

phonometry: standard-conformant acoustic measurement in Python

José Manuel Requena-Plens ¹

¹ Independent researcher, València, Spain

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 27 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

phonometry is a pure-Python library, built on NumPy ([Harris et al., 2020](#)) and SciPy ([Virtanen et al., 2020](#)), for acoustic measurement, analysis and prediction. It covers sound level metrology, fractional-octave filtering, psychoacoustics, room and building acoustics, acoustic materials, human vibration, environmental and transport noise, underwater acoustics, electroacoustics and wave simulation. Every result is a typed, frozen dataclass that carries the inputs it was computed from, draws its own figure, and, for the metrics that certify or rate something, renders an accredited-style report.

What distinguishes the library is not the breadth of that list but how each entry on it is built and verified. Every metric is implemented from the text of its governing standard, clause by clause, rather than from a secondary description or another implementation. Where a standard publishes worked examples, tolerance tables or nominal responses, those numbers are transcribed into the test suite as oracles: the IEC 61672-1 weighting tolerances ([International Electrotechnical Commission, 2013](#)), the IEC 61260-1 filter class limits ([International Electrotechnical Commission, 2014](#)), the ISO 532-1 loudness test signals ([International Organization for Standardization, 2017](#)), the ISO 717 rating examples ([International Organization for Standardization, 2020](#)), and hundreds more. At the time of writing, continuous integration enforces 427 numerical conformance checks against 278 referenced standards on every change, and the per-check report (standard and clause, normative expected value, computed value, delta, verdict) is published with the documentation (<https://jmrplens.github.io/phonometry/reference/conformance/>), alongside a bibliography listing all 278 referenced standards with their editions (<https://jmrplens.github.io/phonometry/reference/bibliography/>). A regression that moves a computed value outside a standard's acceptance limit fails the build, so the conformance claim cannot silently stop being true between releases. [Figure 1](#) shows one such check as the user sees it: the realized A and C frequency weightings against the class limits of IEC 61672-1:2013 Table 3.

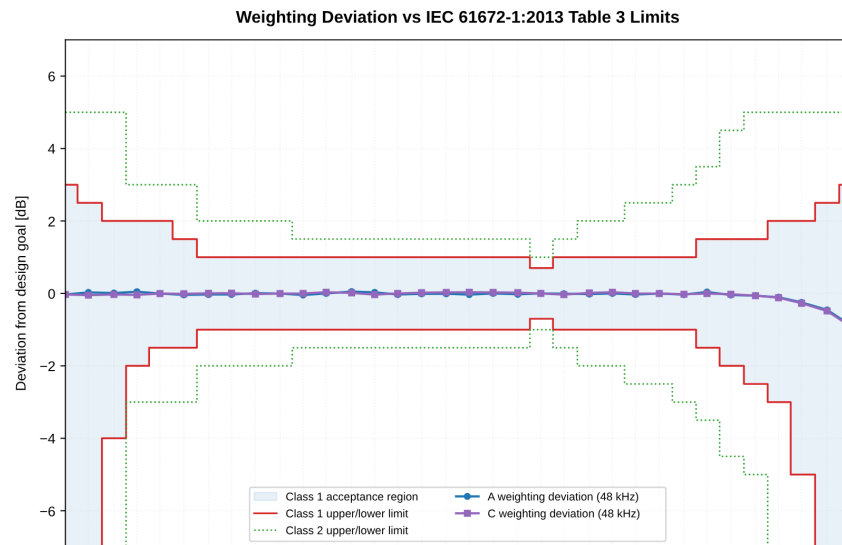


Figure 1: Deviation of the library's A and C frequency weightings (48 kHz) from the design goal, against the class 1 and class 2 acceptance limits of IEC 61672-1:2013 Table 3. The same tolerance table is transcribed in the test suite, so this figure and the CI gate share their oracle.

Statement of need

Acousticians, noise consultants and researchers routinely have to defend a number against a tolerance table: a certified sound level meter quantity, a weighted insulation rating, a declared sound power level. Implementations of the same standardized metric disagree more often than practitioners expect, and when they do, the user is left to guess which one the standard would side with, because the verification evidence behind each implementation is typically partial, informal or unpublished.

phonometry exists to close that gap. It grew out of the author's work as an acoustics researcher: the clause-by-clause readings, the measurement practice and the standards library behind the implementations predate the package, which translates that accumulated domain knowledge into verified code and didactic documentation. Its contribution is a method as much as a library: implement from the primary normative source, transcribe the standard's own reference values as test oracles, and publish the full conformance evidence. Where a standard publishes no numerical example, the implementation is anchored to the closed-form expressions of the normative text and pinned with a synthesized case; the conformance report labels these weaker anchorings as such instead of rounding them up. The library targets practitioners who need defensible numbers (measurement against IEC 61672-1 and IEC 61260-1, ratings under ISO 717 and related building-acoustics standards, environmental indicators under ISO 1996), educators who want each quantity traceable to its defining clause, and researchers who need a verified baseline against which to compare new methods.

State of the field

The scientific-Python acoustics ecosystem is strong on foundations and thin on normative verification. `python-acoustics` ([python-acoustics contributors, 2023](#)) long served as the general-purpose toolbox for acousticians, covering weighting, octave bands and several ISO calculations, but its repository is archived and unmaintained. `pyfar` ([The pyfar developers, 2025](#)) is an excellent, actively developed family of packages for acoustics research (signal classes, DSP, I/O of measurement formats), designed as a foundation rather than as an implementation of the standardized metric catalogue. Specialised packages validate individual

metrics against their reference data, as MOSQUITO (San Millán-Castillo et al., 2021) does for its loudness implementations against the ISO 532 test signals.

Contributing the conformance method to one of these projects was considered and rejected for a scope reason: per-check normative verification only pays off when it is systematic, across every metric a library ships and enforced in its own continuous integration, and none of the existing projects is structured around that contract. What no library in this space publishes is conformance evidence that is complete (every shipped metric), granular (per check, with the normative expected value and the delta) and live (regenerated and enforced on every change), so that the verification cannot drift from the code it describes. That is the niche phonometry occupies; for signal-processing foundations it happily coexists with pyfar.

Software design

Three design decisions carry the method. First, every computation returns a typed, frozen dataclass rather than a bare array: the result object carries the inputs and parameters it was computed from, exposes a `plot()` method that draws the conventional figure for that quantity, and, for metrics whose standards define a reporting format (insulation ratings, absorption classes, declared noise emission), a `report()` method that renders an accredited-laboratory-style fiche citing the governing clauses (Figure 2). The trade-off is verbosity against provenance: a frozen result is a self-documenting evidence trail, which is what a defensible number needs, at the cost of one attribute access compared with returning arrays.

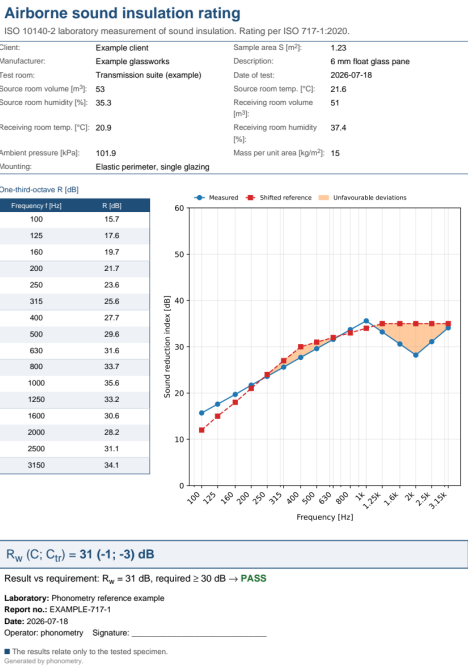


Figure 2: The `report()` method of an airborne insulation result: a single-page fiche in the layout of an accredited laboratory report, stating the measured curve, the shifted reference curve, the weighted rating and spectrum adaptation terms, and the ISO 717-1 clauses each value comes from.

Second, the implementation is pure Python on NumPy and SciPy, with optional Numba

80 acceleration for the heaviest kernels and block-processing entry points for signals that do not
81 fit in memory. A compiled core would be faster still, but clause-by-clause auditability of the
82 numerical code, the property the whole project trades on, favours readable Python whose lines
83 can be mapped to the formulas they implement.

84 Third, the oracles live inside the test suite, not in a separate validation document. Standards'
85 worked examples and tolerance tables are transcribed as test expectations, and the published
86 conformance report is generated from the same run that gates merges. The alternative, a
87 hand-maintained validation report, was rejected because it can silently drift from the code; here
88 the report cannot exist in a state the tests did not produce. The library is organised in twelve
89 domain namespaces over a flat API that remains importable directly, and the documentation, in
90 English and Spanish, pairs every metric with a guide that teaches the method and a reference
91 page that cites the exact clauses and formula numbers implemented; machine-readable copies
92 of every page are published alongside the rendered site.

93 Research impact statement

94 The library has seen research use under its previous name, PyOctaveBand, released in 2020
95 and downloaded and integrated by third parties since; a published example is the DNN-based
96 hearing-aid processing line of work of Drakopoulos & Verhulst (2023), whose released models
97 use it for octave-band analysis. A second, distinctive form of impact runs in the other direction:
98 re-deriving formulas and recomputing worked examples from the source documents has surfaced
99 defects in the published standards themselves (misprints, worked examples that contradict
100 their own normative clauses, broken cross-references). Each confirmed case is documented in
101 a public errata registry with the evidence, the reading the library implements, and whether it
102 has been reported to the issuing body, so the verification work feeds back into the standards
103 it verifies. The per-check conformance report also gives researchers a citable, reproducible
104 baseline: a paper comparing a new loudness or insulation model against phonometry can point
105 to the exact normative checks its baseline passes.

106 AI usage disclosure

107 Generative AI tools (large language models) were used as an implementation assistant
108 throughout the development of the library, its documentation and this paper: drafting code
109 and tests from the author's clause-by-clause readings of the normative texts and drafting
110 documentation prose, always under the author's direction and review. The domain knowledge
111 the library encodes, the normative readings, the measurement practice and the resolution
112 of ambiguities in the texts, comes from the author's prior research work in acoustics. The
113 verification framework this paper describes is also the mechanism by which AI-assisted output is
114 controlled: every metric, however drafted, must reproduce the standard's own worked examples
115 and tolerance tables in the conformance suite before it ships, and the per-check report of
116 that suite is public. The core design decisions are the author's: the normative-source-only
117 implementation policy, the oracle-transcription method, the result-object architecture and the
118 errata registry. All content, including AI-assisted content, was reviewed and approved by the
119 author.

120 Acknowledgements

121 phonometry was first released in 2020 as PyOctaveBand, an octave-band filtering package;
122 the project was renamed when its scope outgrew the name. The author thanks the users
123 who reported numerical discrepancies over the years (several of the conformance checks exist
124 because someone asked why two implementations disagreed), and the colleagues of his research
125 years at the Universitat Politècnica de València, at the Gandia campus (EPSG) and at the

126 Institute for Instrumentation in Molecular Imaging (i3M, UPV-CSIC), where the measurement
127 practice behind this library was formed. This work received no external funding.

128 References

- 129 Drakopoulos, F., & Verhulst, S. (2023). A DNN-based hearing-aid strategy for real-time
130 processing: One size fits all. *Proceedings of the IEEE International Conference on Acoustics,*
131 *Speech and Signal Processing (ICASSP 2023)*. [https://doi.org/10.1109/ICASSP49357.](https://doi.org/10.1109/ICASSP49357.2023.10094887)
132 [2023.10094887](https://doi.org/10.1109/ICASSP49357.2023.10094887)
- 133 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,
134 Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk,
135 M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant,
136 T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
137 [/doi.org/10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2)
- 138 International Electrotechnical Commission. (2013). *IEC 61672-1:2013, Electroacoustics. Sound*
139 *level meters. Part 1: Specifications*. International Electrotechnical Commission, Geneva.
- 140 International Electrotechnical Commission. (2014). *IEC 61260-1:2014, Electroacoustics.*
141 *Octave-band and fractional-octave-band filters. Part 1: Specifications*. International
142 Electrotechnical Commission, Geneva.
- 143 International Organization for Standardization. (2017). *ISO 532-1:2017, Acoustics. Methods*
144 *for calculating loudness. Part 1: Zwicker method*. International Organization for
145 Standardization, Geneva.
- 146 International Organization for Standardization. (2020). *ISO 717-1:2020, Acoustics. Rating of*
147 *sound insulation in buildings and of building elements. Part 1: Airborne sound insulation*.
148 International Organization for Standardization, Geneva.
- 149 python-acoustics contributors. (2023). *python-acoustics: A Python library aimed at*
150 *acousticians*. <https://github.com/python-acoustics/python-acoustics>.
- 151 San Millán-Castillo, R., Latorre-Iglesias, E., Glessner, M., Wanty, S., Jiménez-Caminero, D.,
152 & Álvarez-Jimeno, J. M. (2021). MOSQUITO: An open-source and free toolbox for sound
153 quality metrics in the industry and education. *Proceedings of the INTER-NOISE and NOISE-*
154 *CON Congress and Conference (INTER-NOISE 2021)*, 263. [https://doi.org/10.3397/IN-](https://doi.org/10.3397/IN-2021-1767)
155 [2021-1767](https://doi.org/10.3397/IN-2021-1767)
- 156 The pyfar developers. (2025). *pyfar: Python packages for acoustics research*. [https://pyfar.](https://pyfar.org/)
157 [org/](https://pyfar.org/).
- 158 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
159 Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M.,
160 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ...
161 SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing
162 in Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>